SDP Research Report - Template

Project Name	ShadowTrace
Faculty Supervisor Name	Dr. John Burns
Team Lead Name & ID	Said Mehraliyev 000014989
Team member #1 Name & ID • What sections did this person contribute to?	Said Mehraliyev 000014989
Team member #2 Name & ID What sections did this person contribute to?	Jamil Akbarov 000015227
Team member #3 Name & ID • What sections did this person contribute to?	Ibrahim Najafguliyev 000019888

Word count guide: 2000-2500
Table of contents

1. Introduction
    1. Background and Context
    2. Problem Statement / Research Questions and Hypotheses
    3. Proposed Solution Objectives
    4. Impact of the Proposed Solution

2. Review
    1. Overview of Existing Market
    2. Gaps in Current Market

3. Proposed Solution
    1. Project Goals – list and discuss each use case
    2. System Architecture and Design
    3. Technologies and Tools
    4. Data Collection and Analysis Methods (if applicable)
    5. Evaluation Metrics and Success Criteria

4. Expected Outcomes
    1. Anticipated Deliverables
    2. Potential Impact and Applications

5. Ethical Considerations (if applicable)
    1. Data Privacy and Security
    2. Ethical Implications of the Project

6. Conclusion

1. Summary of Proposal

7. References
    1. List of all referenced resources using IEEE reference standards (ref https://ieee-dataport.org/sites/default/files/analysis/27/IEEE%20Citation%20Guidelines.pdf)
    2. Appendices (if applicable)
    3. Supplementary materials (e.g., preliminary code snippets, diagrams, or datasets)

1. Introduction

Background and Context

Software engineering field has shifted monolithic applications toward microservice architectures. These systems decompose complex software into loosely coupled, independently deployable services communicating over lightweight network protocols such as gRPC, HTTP, and REST. For the maintenance of operational visibility, organizations deploy distributed tracing infrastructures, with OpenTelemetry (OTel) serving as the Cloud Native Computing Foundation (CNCF) industry standard. Distributed tracing records the execution path of a request as it traverses service boundaries. Each unit of work is captured as a "span," and the complete execution path is structured as a Directed Acyclic Graph (DAG), referred to as a "trace."

Problem Statement / Research Questions and Hypotheses

Problem Statement

While distributed tracing provides observability, most tools used within the industry expose a critical security vulnerability known as the Privacy-Security Paradox. OpenTelemetry SDKs capture HTTP headers, database queries, URL parameters, and payload attributes by default. Consequently, telemetry streams routinely log sensitive data, including passwords, JWT authorization tokens, session keys, credit card numbers, and Personally Identifiable Information (PII), all in plain text.

Conversely, attempting to secure microservices using current anomaly detection methods introduces a conflict. Recent research by Yunjian Ma (May 2026) demonstrated that graph neural networks relying solely on static request-level call-path structures fail to detect quiet application-layer attacks, achieving a Macro F1 score of only 0.405. To achieve high detection accuracy (F1 = 0.955), existing systems are forced to ingest unredacted log text payloads (using TF-IDF features) and performance metrics.

Research Questions

RQ1: To what extent do default OpenTelemetry auto-instrumentation pipelines leak sensitive PII and authentication credentials across microservice benchmark suites?

RQ2: Can a Spatio-Temporal Graph Neural Network (ST-GNN), implemented on redacted call-path topologies and time-weighted edge latencies, achieve detection accuracy comparable to log-dependent baselines for application-layer lateral movement?

RQ3: What is the quantitative Pareto trade-off between the degree of telemetry attribute redaction (privacy) and intrusion detection accuracy (security utility)?

Hypotheses

H1: Static trace topology alone is insufficient for intrusion detection because semantically malicious requests (e.g., brute-force logins or lateral movement hops) often mirror valid two-node execution paths.

H2: Coupling structural call-paths with inter-span latency distributions (T_child – T_parent) and temporal sequence windowing provides a sufficient behavioral signature to classify application-layer attacks without reading payload attributes.

2. Review

2.1 Overview of Existing Market

Observability and APM Platforms (e.g., Datadog, Dynatrace, Splunk, Jaeger):

These platforms rely on standard telemetry frameworks like OpenTelemetry (OTel) to collect metrics, logs, and distributed traces across microservices. By default, auto-instrumentation SDKs capture HTTP request headers, URL query parameters, database queries, and error stack traces. Data is transmitted over the network and stored in centralized databases in unencrypted or plain-text formats. While this provides deep visibility for performance debugging, it routinely logs sensitive data—including passwords, JWT authorization tokens, session keys, and Personally Identifiable Information (PII)—creating significant GDPR and CCPA compliance risks.

Traditional Security & Intrusion Detection Systems (e.g., WAFs, Cloud-Native IDSs):

Traditional security tools rely on inspecting deep packet payloads or reading system logs to detect malicious activity. They search for known signatures, regular expressions, or anomalous keyword patterns within HTTP request bodies. However, this approach creates an inherent conflict: to detect threats, the security system must inspect unredacted, sensitive payload data.

2.2 Gaps in Current Market

The Privacy Gap in Existing Tools: Where privacy features exist in current market tools, they rely almost exclusively on static, post-storage regular expressions (regex). These filters run after data has already been transmitted across the network and stored in centralized databases, leaving telemetry exposed to interception during transit. Furthermore, regex filters frequently miss novel, unstructured, or high-entropy secrets (such as API keys or OAuth tokens).

The Specific Gap in Yunjian Ma's (2026) Baseline: Recent research by Yunjian Ma (May 2026) evaluated whether graph neural networks (GCNs) could serve as a foundation for microservice

intrusion detection. Ma demonstrated a fundamental limitation in current graph-based detection: a GCN relying solely on static, request-level trace structure achieves a Macro F1 score of only 0.405, failing to detect quiet application-layer attacks like brute-force logins or lateral movement hops. This occurs because semantically malicious requests often follow the exact same two-node call path (Frontend→Auth) as legitimate user requests.

The Core Limitation: To achieve high accuracy (F1 = 0.955), Ma's approach was forced to ingest 16-dimensional log TF-IDF text features and 9-dimensional performance metrics for every service node. This proves the core market gap: existing systems cannot detect silent intrusions without ingesting unredacted, privacy-invasive log payloads.

3. Proposed Solution

3.1 Project Goals – Use Case Discussion

ShadowTrace realizes four primary operational use cases:

1. **Use Case 1 (In-Stream Redaction):** The OTel Collector Processor calculates Shannon Entropy (H) on incoming span attributes. High-entropy attributes ($H \geq 3.5$) are redacted in memory before storage.
2. **Use Case 2 (Metadata-Blind Intrusion Detection):** The ST-GNN receives redacted graph sequences and classifies application-layer lateral movement based on call topology and latency timing.
3. **Use Case 3 (Closed-Loop Enforcement):** Detections with confidence > 0.95 trigger the automated generation of Istio AuthorizationPolicy YAML files, blocking unauthorized service-to-service paths in real-time.
4. **Use Case 4 (Governance Auditing):** Calculates the Telemetry Governance Score (Ts) to quantify privacy risk exposure across microservice namespaces.

3.2 System Architecture and Design

ShadowTrace is structured as a three-layer pipeline:

1. **Privacy Layer (In-Stream Auditor):** A custom Golang processor for the OTel Collector executing real-time Shannon Entropy attribute scrubbing.
2. **Reconstructor Layer (Spatiotemporal Graph Logic):** A stateful Python/Redis sliding-window buffer (t=5s) that stitches span fragments into time-weighted DAGs (Gi).
3. **Analytics Layer (Behavioral ML Engine):** A PyTorch Geometric ST-GNN that evaluates path dispersion, causal history, and temporal jitter to detect security anomalies.

Pipeline Flow:

Microservices / Envoy ---> Stage 1: Go OTel Collector Processor (Entropy Scrubbing) --->

Stage 2: 5s Sliding Window DAG Reconstructor ---> Stage 3: ST-GNN ML Engine ---> Live Security Alert / Istio Block

3.3 Technologies and Tools

1. Target Infrastructure & Microservice Benchmark

Minikube & Docker: We use Minikube to orchestrate a single-node Kubernetes cluster locally on Docker. This gives us a controlled, reproducible environment to run microservice workloads without incurring public cloud infrastructure costs.

Google Online Boutique: To test our framework against real-world microservice behaviors, we deploy Google's industry-standard e-commerce benchmark (11 microservices written in Go, C#, Node.js, Python, and Java). This provides realistic inter-service dependencies, database queries, and multi-tier call paths.

2. Telemetry Transport & In-Stream Edge Processing

OpenTelemetry (OTel) & OTLP: OpenTelemetry provides the vendor-neutral standard for capturing request journeys across our services. The OpenTelemetry Protocol (OTLP) handles data transport over the network, using gRPC (port 4317) for fast inter-service communication and HTTP (port 4318) for web-based trace ingestion.

Golang (Go v1.22+): We chose Go to build the custom processor plugin for the OpenTelemetry Collector. Because the core OTel Collector is written natively in Go, writing our in-stream processor in Go allows us to manipulate trace data directly in memory. This enables real-time Shannon Entropy calculations and attribute redaction at light speed, keeping processing latency well below 5ms per span.

3. Traffic Simulation, Monitoring, & Data Harvesting

k6 Load Testing: We use k6 to run automated load scripts that generate synthetic user traffic (browsing, searching, checking out) alongside controlled application-layer attack scenarios (HTTP floods, brute-force logins, SQL injection probes, SSRF probes, and data exfiltration).

Jaeger (REST API & Storage): During testing, Jaeger provides a visual interface (http://localhost:16686) to inspect call paths. More importantly for our research, Jaeger's REST API (/api/traces) allows our automated harvesting scripts (harvest_traces.py) to extract raw execution graphs to form our offline research datasets.

Prometheus & Grafana: Prometheus scrapes operational metrics from Envoy sidecar proxies, while Grafana visualizes the system's CPU usage, memory footprint, and latency overhead to quantify the performance impact of our security pipeline.

4. Machine Learning & Graph Analytics Engine

Python (v3.11+), PyTorch Geometric (PyG v2.5+), & NetworkX: We use Python for our analytics engine. NetworkX handles the structural formulation, converting raw JSON span relationships into Directed Acyclic Graphs (DAGs). PyTorch Geometric then provides the specialized graph neural network primitives needed to implement spatio-temporal message passing and graph pooling. Scikit-learn and Pandas are used for data scaling and multi-class confusion matrix evaluation.

3.4 Data Collection and Analysis Methods (SAID)

ShadowTrace incorporates three mathematical scoring models to formalize data privacy, risk assessment, and structural graph classification:

1. Shannon Entropy for Privacy Quantification

To automate PII and token detection without relying on static keyword lists, the Privacy Quantifier calculates the Shannon Entropy (H) of every string attribute X present within incoming span metadata:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \backslash log_{2(P(x_i))}$$

Where:

- X is the attribute string value (e.g., HTTP header, URL parameter, or query string).
- n is the total length of string X
- $P(x_i)$ is the probability of occurrence of character x_i within string X.

Redaction Rule: Low-entropy strings (e.g., "status: 200", "method: GET") represent structured, predictable metadata and are passed unaltered. Strings exceeding an entropy threshold τ (e.g., $\tau \geq 3.5$), or matching regex signatures for credit cards/JWTs, represent unstructured identifiers or secret tokens and are redacted in-stream:

$$Action(X) = \begin{cases} X, & if\ H(X) < \tau \\ [\text{REDACTED}], & if\ H(X) \geq \tau \end{cases}$$

2. Telemetry Governance Score (T_s)

To scale privacy enforcement dynamically across a microservice mesh, ShadowTrace calculates a Telemetry Governance Score (T_s) for each service span:

$$T_s = \sum_{k} \left(H(x_k) \cdot D_{asset} \cdot P_{depth} \right)$$

Where:

- $H(x_k)$ is the Shannon Entropy of attribute x_k

- D_{asset} is the Data Asset Sensitivity Score of the originating microservice ($D \in [0, 5]$):
    - $D = 0$: Public Tier (e.g., catalogservice — no sensitive data).
    - $D = 3$: PII/User Tier (e.g., emailservice, cartservice — handles user metadata).
    - $D = 5$: Critical/Financial Tier (e.g., paymentservice, authservice — handles payment/credentials).
- P_{depth} is the Propagation Depth, representing the service's topological distance from public ingress gateways.

3. Time-Weighted Edge Latency Model

Every trace is constructed as a Directed Acyclic Graph $G = (V, E, W)$, where V represents microservice nodes, E represents parent-child invocation calls, and W represents the set of Time-Weighted Edge Latencies (w_{ij}):

$$w_{ij} = T_{end(S_j)} - T_{start(S_i)}$$

Where $T_{\text{start}}(S_i)$ is the timestamp when parent service S_i initiated the call, and $T_{\text{end}}(S_j)$ is the timestamp when child service S_j completed execution.

4. Expected Outcomes

Anticipated deliverables. Four artefacts are expected. First, a redaction component that scores endpoints automatically and replaces sensitive values with keyed tokens, deployed as an OpenTelemetry Collector processor. Second, a detector operating on four payload-free signals, with measured accuracy against a labelled benchmark and a result for the brute-force case that per-request models are documented to miss. Third, a trade-off curve quantifying what minimization costs in detection capability. Fourth, a reference implementation on Kubernetes with reported runtime overhead, alongside the artefacts in Appendix B.

A weak recovery result remains a complete deliverable: a measured ceiling on payload-free detection is useful to anyone designing a minimization-first pipeline, and publication is committed to in either direction.

Potential impact and applications. The immediate application is any organisation running distributed tracing under GDPR, where telemetry has quietly become a second copy of production data. Making minimization architectural rather than retrospective turns a recurring compliance burden into a configured property of the pipeline, and automatic expiry discharges the storage-limitation obligation without operator intervention.

Beyond compliance, the result speaks to a broader question: how much security signal survives aggressive minimization. Existing literature assumes access to payload; this work measures what remains without it. The finding transfers to regulated sectors where payload inspection is restricted by law rather than choice, and to any setting where telemetry crosses a trust boundary.

5. Ethical Considerations

5.1 Data Privacy and Security

The project handles no real personal data. It runs against Google Online Boutique, where catalogue items are fabricated, payment and email are mocked, and all traffic is generated by k6 scripts, so no living person can be identified from any record.

Two design choices carry the privacy claim. First, sensitive values are not stored and later cleaned up — they are replaced at the collector, before anything reaches the database. Second, stored observation windows expire automatically through a time-to-live index, so retention is a property of the data store rather than something a person has to remember.

One qualification matters. The Privacy Layer replaces sensitive values with keyed one-way tokens that stay linkable on purpose, since linkability is what the detection depends on. Under the GDPR this is pseudonymization, not anonymization, and pseudonymous data is still personal data. The no-personal-data position therefore rests on the traffic being synthetic, not on redaction, and would apply differently on live deployment. The signing key is treated as a secret in its own right, since anyone holding it could reverse the mapping.

5.2 Ethical Implications of the Project

The entropy scorer is dual-use by construction. A tool that reliably locates credentials inside telemetry in order to remove them is also a tool that locates credentials. It is run only against the local environment, scoped to redaction rather than extraction, and named openly as such rather than described only in protective language. The sensitivity score behaves similarly, since its blast-radius term effectively ranks which endpoints are most valuable to compromise.

Automated enforcement raises a second concern. Blocking traffic on a detector whose accuracy is still being established risks causing the outage it was meant to prevent, which is why enforcement defaults to dry-run, requires human approval above a severity threshold, and expires automatically.

Finally, results are reported whichever way they fall. A weak detection result still establishes a measured limit on how far payload-free monitoring can go, and privacy figures are always published beside the detection accuracy they cost, so the trade-off is never shown in half.

6. Conclusion

Summary of proposal. This proposal addresses a contradiction at the center of modern observability: the telemetry that makes distributed systems debuggable also makes them a liability, capturing request bodies, headers and queries that no one intended to retain. Existing anomaly detection assumes payload is available; regulation increasingly assumes it is not.

The work asks whether detection survives minimization. Sensitive values are replaced with keyed one-way tokens rather than removed, preserving linkability while destroying recoverability, and detection then operates on structural and temporal signals alone. The contribution is not the claim that this works, but the measurement of what it costs — reported honestly in whichever direction the evidence falls.